# Projet Spring Boot – Frameworks & Composants

Sujet de projet • Architecture applicative, persistance, messagerie asynchrone et design patterns

## 1. Objectif du projet

En groupe de 3 à 4 étudiants, vous devrez concevoir et développer une application Spring Boot autour d'un domaine métier libre.

Le projet doit démontrer votre capacité à construire une architecture applicative complète intégrant plusieurs composants de l'écosystème Spring, des design patterns présentés en cours, ainsi qu'un système obligatoire de gestion des utilisateurs avec production et consommation d'événements asynchrones.

À la fin du projet, vous devrez rendre :

- Le code source complet sur un ou plusieurs repos GitHub (ou autre hébergeur de code).
- Un diagramme de composants.
- Une documentation technique.
- Une démonstration vidéo fonctionnelle.

## 2. Exigences techniques Spring

Votre application devra obligatoirement intégrer les composants suivants :

| Technologie | Exigence |
|---|---|
| Spring Boot | Configuration et démarrage de l'application |
| Spring Web | API REST complète (CRUD) |
| Spring Data JPA | Persistance d'au moins 3 entités avec relations |
| Spring Services | Implémentation de la logique métier |
| Spring IoC / DI | Injection de dépendances entre composants |
| Spring JMS | Production et consommation de messages asynchrones |
| Base de données | PostgreSQL, MySQL, H2 ou équivalent |

L'utilisation d'une architecture en couches est obligatoire : Controller → Service → Repository → Database. Le module de messagerie devra être intégré à cette architecture.

## 3. Module obligatoire : gestion des utilisateurs et événement JMS

Chaque projet doit obligatoirement intégrer un module de gestion des utilisateurs.

### 3.1 Persistance

Créer une entité User persistée via Spring Data JPA. Cette entité contiendra au minimum : id, name, email, createdAt.

### 3.2 API REST

| Méthode | Endpoint | Fonction |
|---|---|---|
| POST | /users | Créer un utilisateur |
| GET | /users | Lister les utilisateurs |
| GET | /users/{id} | Consulter un utilisateur |
| PUT | /users/{id} | Modifier un utilisateur |

| | | |
|---|---|---|
| DELETE | /users/{id} | Supprimer un utilisateur |

### 3.3 Publication d'un événement JMS

Lorsqu'un utilisateur est créé avec succès, l'application doit publier un message JMS dans une queue.

Le message doit contenir au minimum : l'identifiant de l'utilisateur créé, un timestamp et le type d'événement (USER_CREATED). Des informations complémentaires peuvent être ajoutées (nom, email, etc.).

La publication devra être assurée par un composant Spring dédié, par exemple un service utilisant JmsTemplate.

## 4. Consommation des messages : Queue ou Topic

Chaque projet devra implémenter au moins un consommateur JMS. Deux solutions sont possibles :

- Queue (point-à-point) : chaque message est traité par un seul consommateur.
- Topic (publish-subscribe) : un message peut être distribué à plusieurs abonnés.

Le consommateur devra réagir à la création d'un utilisateur et réaliser un traitement métier identifiable, par exemple une notification interne, un enregistrement dans un journal d'audit ou une mise à jour d'un autre module.

L'utilisation de @JmsListener est recommandée. Le fonctionnement asynchrone devra être visible lors de la démonstration.

## 5. Design Patterns – Contraintes et liberté

Vous devrez implémenter et justifier au minimum 3 design patterns parmi ceux étudiés en cours. Les étudiants sont libres de choisir les patterns adaptés à leur domaine métier.

| Design Pattern | Exemple d'application dans Spring |
|---|---|
| Event Message | Publication d'un événement USER_CREATED |
| Content-based Router | Routage d'un message selon son type |
| Message Filter | Ignorer les événements ne répondant pas à certaines conditions |
| Message Translator | Transformation d'un objet métier en message |
| Content Enricher | Ajout d'informations complémentaires à un événement |
| Event-driven Consumer | Traitement automatique par un @JmsListener |
| Lazy Load | Chargement différé des relations JPA |
| Embedded Value | Utilisation de @Embeddable et @Embedded |

Les patterns devront avoir une véritable utilité architecturale ou métier. Leur simple mention dans la documentation ne sera pas suffisante. Un pattern supplémentaire, correctement implémenté et justifié, pourra donner lieu à un bonus.

## 6. Modélisation attendue

Vous devez fournir un diagramme de composants représentant :

- Les contrôleurs REST.
- Les services métier Spring.
- Les repositories Spring Data JPA.
- Les principales entités et la couche de persistance.
- Le producteur et le consommateur JMS.
- Le broker de messages.
- Les interactions et les flux de données entre les composants.
- L'emplacement des design patterns retenus.

Le diagramme doit notamment permettre de comprendre le cheminement d'une requête depuis le contrôleur REST jusqu'au traitement asynchrone d'un événement.

## 7. Documentation attendue

1. Une présentation du problème métier et des fonctionnalités de l'application.
2. Une explication de l'architecture Spring et du rôle des différents composants.
3. Le diagramme de composants global.
4. La description du module de gestion des utilisateurs.
5. La présentation des design patterns retenus et leur justification.
6. Un scénario d'utilisation complet illustrant le flux REST → Controller → Service → Repository/JPA → JMS → Consumer.
7. Un résumé des contributions individuelles des membres du groupe.
8. Les instructions de configuration, de lancement et de test de l'application.

## 8. Exemples de domaines possibles

Le domaine métier reste libre, sous réserve de permettre l'implémentation des composants demandés.

- Plateforme de réservation de salles ou d'équipements.
- Application de gestion d'événements.
- Système de gestion des stocks.
- Plateforme de suivi de commandes.
- Application de suivi sportif.
- Gestionnaire de tournoi e-sport.
- Plateforme de gestion de missions spatiales.

## 9. Critères d'évaluation (20 points)

| Critère | Points |
|---|---|
| Gestion des utilisateurs (API REST + JPA) | 2 |
| Production JMS à la création d'un utilisateur | 2 |
| Consommateur JMS fonctionnel | 2 |
| Autres composants Spring et fonctionnalités métier | 5 |
| Design patterns : implémentation et pertinence | 4 |
| Qualité de la documentation | 2 |
| Qualité du diagramme de composants | 1 |
| Organisation du code et démonstration | 2 |
| Total | 20 |
| Bonus technique éventuel | +1 |

## Annexe 1 – Structure recommandée pour la documentation

9. Présentation générale de l'application.
10. Diagramme de composants.
11. Description des couches Spring (Controller, Service, Repository, Messaging).
12. Description du module utilisateur.
13. Présentation des design patterns et justification.
14. Description d'un scénario complet de publication et de consommation JMS.
15. Contributions individuelles des membres du groupe.

## Annexe 2 – Design Patterns autorisés

- Messaging Patterns : Command Message, Document Message, Event Message, Correlation Identifier, etc.

- Routing Patterns : Content-based Router, Message Filter, Splitter, Aggregator, Routing Slip, etc.
- Transformation Patterns : Message Translator, Content Enricher, Normalizer, etc.
- Endpoint Patterns : Messaging Gateway, Event-driven Consumer, Competing Consumers, etc.
- Channel Patterns : Point-to-Point Channel, Publish-Subscribe Channel, Dead Letter Channel, etc.
- Persistence Patterns : Unit of Work, Lazy Load, Embedded Value, stratégies d'héritage JPA, etc.

## Annexe 3 – Recommandations techniques

| Composant | Recommandation |
|---|---|
| Framework | Spring Boot 3.x ou version plus récente compatible, avec Spring Web, Spring Data JPA et validation |
| Broker JMS | Apache ActiveMQ Artemis |
| Base de données | PostgreSQL ou H2 |
| Tests d'API | Postman ou Swagger UI |
| Déploiement local (facultatif) | Docker Compose |

Une attention particulière devra être portée à la cohérence entre la persistance de l'utilisateur et la publication de son événement JMS, notamment en cas d'échec d'une opération.